FPT UNIVERSITY
DEPARTMENT OF SOFTWARE ENGINEERING

RESEARCH PROPOSAL

Research title:	A Comparative Study of GenAI-Generated and Human-Authored Black-Box Test Suites
Sub-committee:	Software Testing
Group name:	SE1907 – Group 1
Authors:	Đặng Phước Vinh, Vũ Hà Ngọc Huyền, Phạm Nguyễn Quyền
Mentor:	Nguyễn Nguyên Bình

Abstract

Black-box testing is a critical quality assurance practice, yet it remains labor-intensive, error-prone, and heavily dependent on tester expertise. This study conducts a rigorous empirical comparison of test suites generated by state-of-the-art Large Language Models (LLMs) against human-authored suites, organized as two methodologically distinct cohorts that are analyzed and reported *separately* throughout. The **reproduced cohort** covers the three original applications of Kirinuki & Tanno [1] (Password Strength Checker, Unit Converter, Budget Planner): the published human (Human A–D) and ChatGPT (GPT-4) test suites are reused unchanged as a fixed baseline, and this study's own contribution is generating three *additional* LLM suites (GPT-5.5, Claude 3.5 Sonnet, Gemini 3.1) against that same baseline. The **new cohort** covers two CLI applications authored specifically for this study (Library Fine Calculator, Task Manager): here both the AI suites (Sonnet 4.6, Gemini 3.1, ChatGPT-5.5) and the human suites (three third-year software engineering students, testing independently without AI assistance) are entirely original data collected by this team. Test viewpoints were manually extracted and mapped into a unified coverage matrix (`test perspective analysis 2.xlsx`), and coverage was measured using five quantitative metrics. Statistical significance was validated with an exact two-sided Wilcoxon signed-rank test computed via dynamic programming. Results show that modern LLMs significantly outperform human testers in viewpoint coverage (exact $p = 0.00002$ pooled across all categories), yet human testers still contribute unique boundary viewpoints that no AI model found. The Human+AI combined union achieves 98.1%–100% coverage across all five applications, confirming a strong synergistic potential for collaborative QA workflows. We conclude by proposing a two-phase QA process — LLM-generated baseline followed by targeted human review — as a practical adoption path for AI-assisted testing.

Keywords: Generative AI, Large Language Models, Software Testing, Black-Box Testing, Test Case Generation, Empirical Comparison, Test Viewpoint Coverage, Wilcoxon Signed-Rank Test.

Contents

1 Introduction

1.1 Background

Black-box testing (specification-based testing) is a foundational software quality assurance (QA) practice in which test cases are designed purely from behavioral specifications, without knowledge of the internal implementation. This approach ensures that all externally observable behaviors — including boundary values, invalid inputs, error handling, and state transitions — are exercised by the test suite.

Traditionally, black-box test design has been entirely manual. Experienced testers analyze requirements documents, identify equivalence classes, boundary conditions, and error scenarios, then craft test cases accordingly. This process is time-consuming: in our study, the four professional developers used as the human baseline spent an average of **197.75 minutes** across the three replicated applications (60.25 min for Password Checker, 63.25 min for Unit Converter, 74.25 min for Budget Planner), yet still missed a significant fraction of viewpoints.

The emergence of Large Language Models (LLMs) trained on vast code repositories and natural language corpora has opened an unprecedented opportunity: these models can read a requirements specification and instantaneously generate comprehensive, structured test cases in natural language. The central question is no longer *can* LLMs generate test cases, but *how well* do they perform compared to humans across diverse applications and different levels of tester experience?

Problem statement

Three core problems motivate this research:

1. **Stale benchmarks on older models.** The baseline study by Kirinuki & Tanno [1] benchmarked ChatGPT (GPT-4) only. Newer-generation models (GPT-5.5, Gemini 3.1, Claude 3.5 Sonnet) have not been rigorously benchmarked against identical human baselines under the same specifications and viewpoint-extraction methodology.

2. **Undifferentiated human baselines.** Most comparative studies treat all human testers as a single "human" baseline. In reality, a professional developer with years of QA experience and a third-year student taking their first software testing course have fundamentally different cognitive testing strategies. Conflating these two groups obscures important insights.

3. **Unmeasured synergy.** Even when studies show that LLMs achieve higher individual coverage than humans, they rarely quantify whether the two sources complement each other. If LLMs and humans cover *different* blind spots, a combined Human+AI suite could achieve near-perfect coverage — the most practically important finding for QA teams deciding how to adopt GenAI.

Research Aim

This study aims to empirically evaluate and statistically validate the black-box test coverage capabilities of modern GenAI LLMs versus human testers across five software specifications, and to measure the synergistic gain achieved by combining their test suites.

Research Objective

- **O1a (reproduced cohort):** Generate three additional LLM test suites (GPT-5.5, Sonnet 4.6, Gemini 3.1) for the three original applications, and merge them into the same viewpoint matrix as the unmodified, published Human A–D and ChatGPT (GPT-4) suites from Kirinuki & Tanno [1] — no new human data is collected for this cohort.
- **O1b (new cohort):** Author two original CLI specifications, generate test suites from three LLMs (Sonnet 4.6, Gemini 3.1, ChatGPT-5.5), and collect entirely new, independently-authored test suites from three third-year students (Student A–C) — both the AI and human data for this cohort are primary data collected by this team.
- **O2:** Extract, categorize, and map all test viewpoints into a unified coverage matrix, keeping the reproduced and new cohorts in clearly separated sheets/sections at every stage.
- **O3:** Compute five quantitative metrics per tester per application: viewpoint coverage, category coverage, missing rate, unique contribution, and Human–AI union — reported per cohort, never pooled across cohorts.
- **O4:** Validate the statistical significance of AI vs. human coverage differences via the exact Wilcoxon signed-rank test.
- **O5:** Analyze the student vs. professional performance gap and its implications for AI adoption in test automation workflows.

2 Literature Review

Overview of Existing Research

Our review followed the PRISMA (Preferred Reporting Items for Systematic Reviews and Meta-Analyses) framework as introduced in the course methodology, searching arXiv, IEEE Xplore, and the ACM Digital Library with terms including *LLM test generation*, *black-box testing*, *requirement-based test case*, and *AI vs. human test suite*. Candidate papers were processed through the four PRISMA stages — Identification, Screening, Eligibility, and Included — before being retained as evidence for the final research design.

Table 1: Selected related work

Paper	Year	Method	Key Finding	Human Baseline
Kirinuki & Tanno [1]	2024	GPT-4, viewpoint coverage	ChatGPT covers basic viewpoints; professionals catch unique boundary cases	Yes (4 professionals)
Arora et al. [2]	2024	RAG + LLM (industrial)	RAG improves domain accuracy of NL test scenarios	No (expert survey only)
Sami et al. [3]	2024	React+Flask+ OpenAI tool	Automates Gherkin writing; struggles with complex state logic	No
Tufano et al. [4]	2024	TestGenEval benchmark	Best model (GPT-4o) averages only 35.2% statement coverage on real repos	Yes (repo history)
Ferreira et al. [5]	2025	AutoUAT + TestFlow (Cypress)	Helpful 95% of the time; edge cases often missed	Yes (QA team)

Research Gaps

- **Gap 1 (Model recency):** No study has benchmarked GPT-5.5, Gemini 3.1, and Claude 3.5 Sonnet together against a consistent human baseline under a reproducible viewpoint-extraction methodology.
- **Gap 2 (Baseline heterogeneity):** Existing studies do not separate the student tester cohort from the professional cohort, masking variance in human test quality that is critical for practical QA decision-making.
- **Gap 3 (Synergy quantification):** No study computes the union (Human+AI combined) coverage as a primary metric to measure how much the combined suite closes the gap toward 100% coverage.

3 Research Scope

Research Questions

- **RQ1:** How does the test viewpoint coverage (%) of state-of-the-art LLMs compare to that of professional developers and software engineering students?
- **RQ2:** Is the difference in viewpoint coverage between AI and human test suites statistically significant, and at what granularity (application level vs. category level)?
- **RQ3:** To what extent do LLMs and human testers complement each other — i.e., what is the Human+AI combined union coverage?

Scope of the Study

Table 2: Scope of the study

Dimension	Details
Applications (5)	Password Strength Checker, Unit Converter, Budget Planner (reproduced cohort); Library Fine Calculator CLI, Task Manager CLI (new cohort)
LLM models	Reproduced cohort: GPT-4 (reused) + GPT-5.5, Sonnet 4.6, Gemini 3.1 (newly generated). New cohort: Sonnet 4.6, Gemini 3.1, ChatGPT-5.5 (all newly generated)
Professional cohort (reused)	Human A–D — published test suites from Kirinuki & Tanno [1], reused unchanged; *not* collected by this team
Student cohort (newly collected)	Student A–C (3rd-year SWE students) — original test suites collected by this team for the 2 new CLI applications
Data source	`test perspective analysis 2.xlsx` — single source of truth for all coverage data, with reproduced and new cohorts kept in separate sheets
Analysis scripts	`analyze_viewpoints.py`, `wilcoxon_test.py`, `author_style.py`, `make_charts.py`

4 Approach and Method

4.1 Research Approach

This is a quantitative empirical study. The methodology involves: (1) systematic test-suite generation and collection, (2) manual viewpoint extraction and categorization, (3) automated coverage computation via Python scripts, and (4) statistical significance testing.

4.2 Data Preparation

Dataset Sources

Five natural language specifications were used as inputs. For the three reproduced applications, the original specifications from Kirinuki & Tanno [1] were used. For the two new CLI applications, original specifications were designed for this study (Section 8).

Data Provenance: Reused vs. Newly Collected

This is the single most important methodological distinction in the study design, and it is kept explicit through every later section, table, and figure:

- **Reproduced cohort (3 sheets — Password Strength Checker, Unit Converter, Budget Planner):** the specifications, the four professional human test suites (Human A–D), and the ChatGPT (GPT-4) test suite are *reused, unmodified,* from the published replication package of Kirinuki & Tanno [1]. This team's contribution for this cohort is limited to *adding* three newly-generated LLM test suites (GPT-5.5, Sonnet 4.6, Gemini 3.1)

against the same specifications, and re-scoring the resulting, extended matrix against the original, untouched human baseline. No new human test data was collected for these three applications.

- **New cohort (2 sheets — Library Fine Calculator, Task Manager):** both the specifications and every test suite — three LLMs *and* three third-year students — are original data produced for this study. This is a self-contained comparison of *new models vs. new (student) human testers*, with no data inherited from prior work.

Because the two cohorts differ in provenance (secondary/reused data vs. primary/newly collected data) as well as in human-tester profile (industry professionals vs. 3rd-year students), they are never pooled into a single "AI vs. human" statistic. Every result table and figure in Section 6 reports the Reproduced and New cohorts as two separate rows/panels, and the Wilcoxon tests in Section 4.4 are likewise computed and interpreted per cohort before any cross-cohort discussion.

All test case files follow a consistent naming convention: `[ModelName]_[AppName]_test_cases.txt`. For example, an excerpt from `GPT-5.5_Budget_planner_test_cases.txt`:

```
# Test case 1
Add income minimum boundary: This test case verify adding an income with
the minimum valid amount.
Test Steps:
- `planner add-income 1 "freelance"`
- Verify that the output is `Income added: $1 from source "freelance".`
```

Prompt Design: Stateful vs. Stateless Applications

Unlike a single generic prompt applied uniformly to every application, this study — following the same design logic as Kirinuki & Tanno [1] — uses *two* zero-shot prompt templates, selected per application according to whether the CLI tool carries internal state across commands. This distinction matters because a *stateless* application (e.g., a password checker) produces one self-contained input/output pair per invocation, whereas a *stateful* application (e.g., a budget planner) requires a *sequence* of commands whose final output depends on all prior commands (add an income, add an expense, then check the balance). Using a single-shot "input $\rightarrow$ expected result" format for a stateful application would be unable to express such sequences, so a distinct "Test Steps" format is required. The application-to-prompt mapping used in this study (recorded in `spec_use_prompt.txt`) is:

Table 3: Prompt template assigned to each specification

Specification	App type	Reason
Password_strength_checker_spec.txt	Stateless	Single password in, single strength verdict out; no data persists between runs.
Unit_converter_spec.txt	Stateless	Single (value, from-unit, to-unit) in, single converted value out.
Library_fine_calculator_cli_spec.txt	Stateless	Each invocation computes one fine from CLI flags; no stored loan records.
Budget_planner_spec.txt	Stateful	Incomes/expenses accumulate across commands; totals depend on the full command history.
Task_manager_cli_spec.txt	Stateful	Tasks persist across add/update/complete/delete commands; correctness depends on state built up over multiple steps.

Stateless prompt template (`prompt_for_stateless_apps.txt`), used for Password Strength Checker, Unit Converter, and Library Fine Calculator:

```
Create at least 50 test cases necessary and sufficient to ensure the correct
behavior of a command line application "{name}" with the following specifications.
For invalid input, an appropriate and descriptive error message you create
should be output.

Your output should be as follows (surround by code block):

# Test case {id}

{Test case name}: This test case verify {description}
Input: `<name> {input}`
Expected result: `{result}`

{specification}
```

Stateful prompt template (`prompt_for_statefull_apps.txt`), used for Budget Planner and Task Manager — identical to the stateless template except that it explicitly resets state at the start of each test case and defers verification to the end of a command sequence:

```
Create at least 50 test cases necessary and sufficient to ensure the correct
behavior of a command line application "{name}" with the following specifications.
It is assumed that no data exists at the start of each test case.
Verify the output only once at the end of the test steps.
For invalid input, an appropriate and descriptive error message you create
should be output.

Your output should be as follows (surround by code block):

# Test case {id}

{Test case name}: This test case verify {description}
Test Steps:
- `<name> {command} {input}`
```

```
- `<name> {command} {input}`
- ...
- Verify that the output is {expected result}.
{specification}
```

The two extra sentences in the stateful template — *"It is assumed that no data exists at the start of each test case"* and *"Verify the output only once at the end of the test steps"* — exist precisely to force each generated test case to be a clean, independently-runnable sequence (avoiding hidden dependencies between test cases) while still allowing the multi-step state build-up that stateful CLI behavior requires. Keeping this distinction identical to the reference paper's own prompt design (rather than collapsing everything into one generic prompt) ensures that any coverage difference observed between applications reflects the LLMs' testing ability, not an artifact of an ill-fitting prompt format.

Inclusion Criteria (Newly Collected Human Test Suites — New Cohort Only)

The following criteria governed the collection of the three student test suites (Student A–C) for the two new applications. They do not apply to the reproduced cohort's Human A–D suites, which were collected under Kirinuki & Tanno's own protocol [1] and are reused as published.

- Human participants must test without access to source code (true black-box setting).
- Test cases must reference at least one expected output from the specification.
- Test suites must be submitted before the AI-generated results are shared.

Exclusion Criteria (All Newly Generated/Collected Suites)

- Test cases that are purely structural (e.g., unit tests referencing internal functions).
- Duplicate test cases within the same test suite.

Data Extraction

For each test suite, every viewpoint was manually extracted and mapped into the Excel matrix. Consistent with the course measurement convention, a viewpoint cell was marked **OK** if the corresponding test suite contained *at least one* test case explicitly addressing that viewpoint; otherwise the cell was left blank. The mapping was cross-reviewed by two researchers to minimize subjectivity bias.

For the three reproduced applications (Password Strength Checker, Unit Converter, Budget Planner), the Excel sheet uses the following column layout:

Table 4: Excel column layout — reproduced applications (15 columns)

Col.	Meaning
A–B	Category, Test Viewpoint
C–I	Human TC#, GPT-5.5, AI TC#, Sonnet 4.6, AI TC#, Gemini 3.1, *(blank)*
J–N	ChatGPT 4.0, Human A, Human B, Human C, Human D
O	Effective viewpoint?

For the two new applications (Library Fine Calculator, Task Manager), only three AI models and three students were included:

Table 5: Excel column layout — new applications (12 columns)

Col.	Meaning
A–B	Category, Test Viewpoint
C–H	TC#, Sonnet 4.6, TC#, Gemini 3.1, TC#, ChatGPT-5.5
I–K	Student A, Student B, Student C
L	Effective viewpoint?

4.3 Proposed Framework / System Design

The analysis pipeline consists of four Python scripts operating on the single Excel data source:

- `analyze_viewpoints.py` → coverage %, union, unique, and miss rate per source per application.
- `wilcoxon_test.py` → exact Wilcoxon signed-rank p-value via dynamic programming.
- `author_style.py` → effective-viewpoint table (Basic vs. Extracted), mirroring the reporting style of Kirinuki & Tanno's Table 2.
- `make_charts.py` → generates Figures 2–6 (cohorts plotted separately; see Section 6).

4.4 Evaluation Method

Coverage, Miss, and Union: Definitions and Intuition

Every viewpoint v in the matrix is a yes/no cell for a given source s (an LLM or a human tester): $covered(v, s) = 1$ if that source's suite contains at least one test case addressing v, and 0 otherwise (Section 4.2, *Data Extraction*). From this binary matrix we define:

$$Cov = \frac{N_{covered}}{N_{total}} \times 100\% \tag{1}$$

$$Cov_{cat} = \frac{N_{covered\,in\,cat}}{N_{total\,in\,cat}} \times 100\% \tag{2}$$

$$Miss = \frac{N_{total} - N_{covered}}{N_{total}} \times 100\% = 100\% - Cov \tag{3}$$

$$Union = \frac{|\{\, v \mid \exists s \in (Human \cup AI),\, covered(v, s) \,\}|}{N_{total}} \times 100\% \tag{4}$$

Eq. (1) is the coverage of a *single* source (e.g., GPT-5.5 alone) over the whole application; Eq. (2) is the same ratio restricted to one category (e.g., only the "Boundary Value" rows), and is what drives the per-category gap in Figure 6. Eq. (3) is simply the complement of coverage and quantifies the risk left unaddressed by a single source. Eq. (4), the *union* coverage, is the most important formula for RQ3: it does not average individual coverages, but instead asks, for each viewpoint, "did *any* source in this group cover it?" — which is why AI union and Human union in Table 8 are always $\geq$ the best individual source in that group. Unique contribution is defined analogously as the count of viewpoints covered by *exactly one* source and by no other source in either group.

Figure 1 illustrates how these quantities partition the full viewpoint set N_{total} for a single application: the AI union and Human union are the two circles, their intersection is the "common core" (covered by both), each circle's exclusive area is that group's unique contribution, and any area outside both circles is the **blind spot** — viewpoints missed by every source, AI and human alike. The Human+AI combined coverage (Eq. (4) applied to $Human \cup AI$) is exactly the area of the two circles together; 100% combined coverage means the blind-spot region is empty.

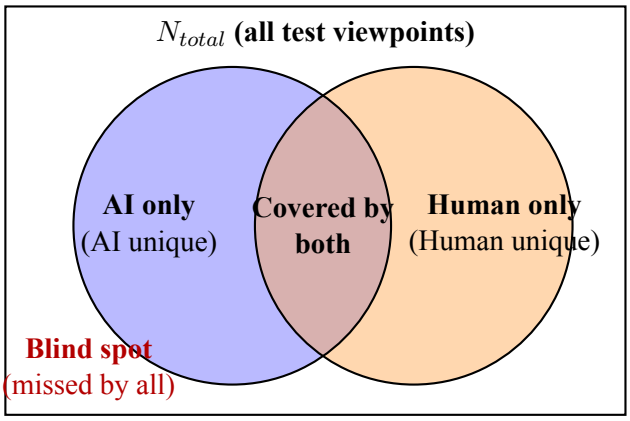

Figure 1: How Cov, Union, unique contribution, and the blind spot partition N_{total} for one application. Compare with the real per-cohort breakdowns in Figures 3 and 5.

The Wilcoxon Signed-Rank Test (Exact p-value via DP)

The Wilcoxon test answers a different question than coverage alone: not "how large is the AI–Human gap on average", but "is that gap *consistently* in the same direction across independent pairs (applications or categories), more than chance would allow". Let $d_i = Cov_{AI,i} - Cov_{Human,i}$ for each paired unit i (an application or a category). After discarding ties ($d_i = 0$), rank $|d_i|$ from smallest to largest (tied absolute differences share the average of their ranks) and compute

$$W^+ = \sum_{d_i>0} \text{rank}(|d_i|), \quad W^- = \sum_{d_i<0} \text{rank}(|d_i|), \quad W = \min(W^+, W^-). \tag{5}$$

Intuitively, if AI and Human coverage were drawn from the same distribution, positive and negative differences should mix evenly, so W^+ and W^- should be close to balanced ($\approx \text{Total}/2$ each); a very small W means the ranks overwhelmingly favor one direction. The exact two-sided p-value is then computed by dynamic programming over all $2^{N_{usable}}$ possible $\pm$ sign assignments of the ranks — i.e., every way the null hypothesis could have produced the observed ranks by chance. Ranks are doubled to integers to handle half-ranks from ties; let r_j be the doubled rank of pair j, so the DP transition is

$$dp[s] \mathrel{+}= dp[s - r_j], \quad \text{for } s = r_j, r_j + 1, \ldots, \text{Total}, \tag{6}$$

where $dp[s]$ counts the number of sign assignments whose negative-rank sum equals s, and the exact p-value is the fraction of all $2^{N_{usable}}$ assignments at least as extreme as the observed W on either tail:

$$p = \frac{\sum_{s \le 2W} dp[s] + \sum_{s \ge \text{Total}-2W} dp[s]}{2^{N_{usable}}}. \tag{7}$$

Worked example (application level, $N = 5$) — reproduces Table 9, row 1 exactly. Using the five application-level averages already reported in Table 8-style data (AI average and Human average per application, from `ket_qua_phan_tich.txt`):

Table 6: Step-by-step Wilcoxon ranking at the application level

Application	AI avg.	Human avg.	d_i (pp)	$\lvert d_i \rvert$	Rank
Budget Planner	78.13%	64.38%	+13.75	13.75	1
Password Strength Checker	75.73%	56.28%	+19.45	19.45	2
Unit Converter	83.65%	62.08%	+21.58	21.58	3
Library Fine Calculator	95.80%	66.67%	+29.13	29.13	4
Task Manager	86.23%	56.33%	+29.90	29.90	5

All five differences are positive and no ties occur, so $W^+ = 1{+}2{+}3{+}4{+}5 = 15$ and $W^- = 0$, giving $W = \min(15, 0) = 0$ — exactly the $W^+ = 15.0$, $W^- = 0.0$ reported for the application-level test in Table 9. Doubling the ranks gives $r = \{2, 4, 6, 8, 10\}$ (Total $= 30$). Because $W = 0$, we need $\sum_{s \le 0} dp[s] + \sum_{s \ge 30} dp[s]$: the only subset of $\{2, 4, 6, 8, 10\}$ summing to 0 is the empty set ($dp[0] = 1$), and the only subset summing to 30 is the full set ($dp[30] = 1$), so the count is $1 + 1 = 2$, and

$$p = \frac{2}{2^5} = \frac{2}{32} = 0.0625, \tag{8}$$

which matches Table 9 exactly. This also demonstrates *why* $p = 0.0625$ is the smallest two-sided p-value obtainable with only 5 paired applications: it is reached whenever all differences point the same direction, regardless of their magnitude, because the DP only has $2^5 = 32$ possible sign patterns to distribute mass over. The category-level tests in Table 9 (Test 2, N=20; Test 3, N=24) follow the identical procedure over category-level d_i instead of application-level d_i, computed automatically by `wilcoxon_test.py`.

4.5 Ethical Considerations

The two cohorts raise different, cohort-specific ethical considerations. For the **new cohort**, all human data from student participants (Student A, B, C) is newly collected by this team and is fully anonymized; no personally identifiable information is stored in the analysis matrix, and participation was voluntary. For the **reproduced cohort**, the professional developer data (Human A–D) and the ChatGPT (GPT-4) suite are not collected by this team at all — they are reused, with attribution, from the already-published baseline study [1], so no new consent or data-collection procedure was required on our part for that cohort. This research uses no sensitive datasets and poses no ethical risks.

5 Research Plan

No.	Task	Expected Output	Members	Target
1	Literature review & PRISMA screening	Finalized literature matrix, 5 selected papers	All	Week 1
2	Specification preparation & Zenodo collection	5 spec files, 3 prompt template files	All	Week 2

No.	Task	Expected Output	Members	Target
3	Generate AI test cases (4 LLMs × 5 apps)	20 AI test case files	All	Week 3
4	Collect human test suites (7 participants)	7 human test case files	All	Week 3
5	Viewpoint extraction & Excel matrix mapping	Completed `test perspective analysis 2.xlsx`	All	Week 4
6	Write & validate Python analysis pipeline	`analyze_viewpoints.py`, `wilcoxon_test.py`, charts	All	Week 5
7	Statistical analysis & interpretation	`wilcoxon_ket_qua.txt`, `ket_qua_phan_tich.txt`	All	Week 5
8	Final report writing (Research Proposal)	Complete research proposal document	All	Week 6
9	Internal review & revision	Revised proposal	All	Week 7

Table 7: Research plan and task allocation

6 Expected Outcomes

The following results were obtained by executing the Python analysis pipeline against `test perspective analysis 2.xlsx`. As established in Section 4.2 (*Data Provenance*), the reproduced cohort (reused Human A–D / GPT-4 baseline) and the new cohort (newly-collected students) are two different comparisons and are therefore never drawn on the same chart — each cohort gets its own pair of figures below, so that a bar chart never visually implies a single, homogeneous 5-application experiment.

Reproduced Cohort (Password, Unit Converter, Budget Planner)

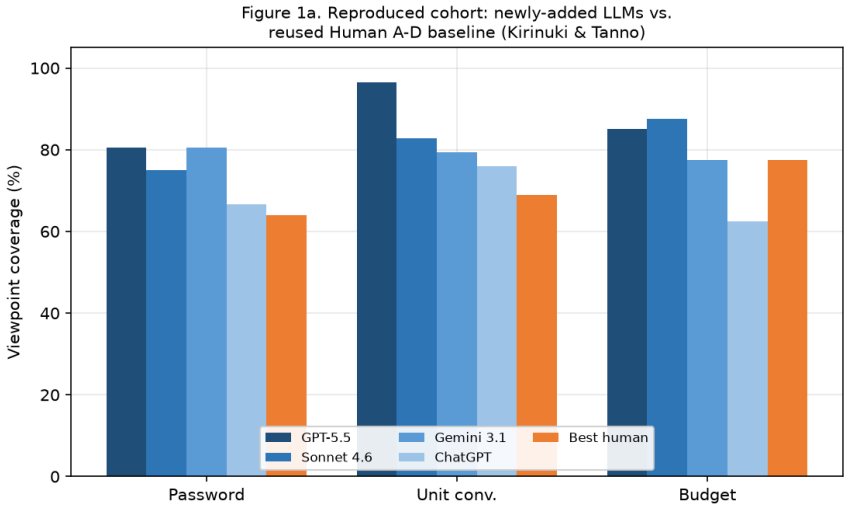

Figure 2: Reproduced cohort — newly-added LLMs (GPT-5.5, Sonnet 4.6, Gemini 3.1) plus the original GPT-4, vs. the best of the reused Human A–D baseline, per application

Figure 2 shows that across the three reproduced applications, the newly-added AI models consistently and substantially outperform the best of the four reused professional testers. The coverage gap is largest in the Unit Converter (GPT-5.5: 96.6%, best human: 69.0%) and smallest in the Budget Planner (Sonnet 4.6: 87.5%, Human D: 77.5%) — an area where systematic enumeration is a known LLM strength.

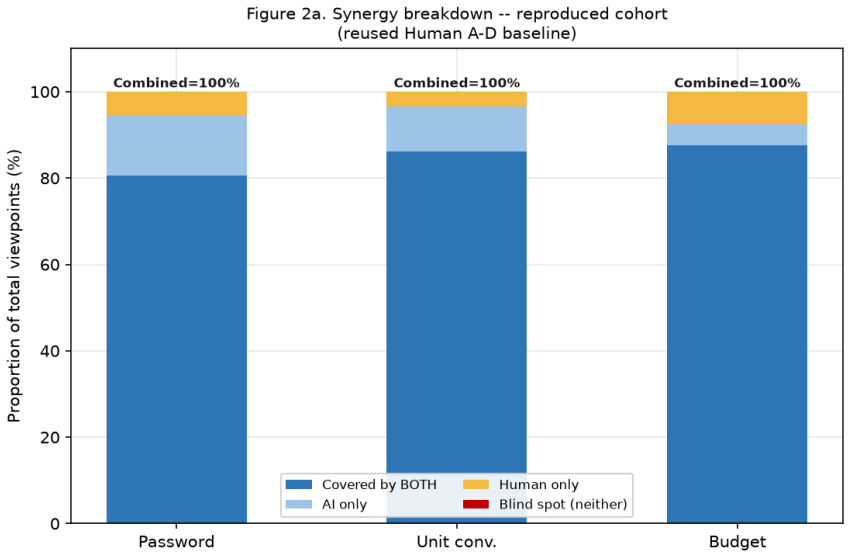

Figure 3: Synergy breakdown — reproduced cohort (reused Human A–D baseline)

Figure 3 uses a stacked bar chart with four segments per application: **covered by both** (the common core), **AI only**, **human only**, and **blind spot** (missed by everyone). Key observations for this cohort:

1. **100% combined union:** In the Password Strength Checker and Unit Converter, the combined (newly-generated AI) ∪ (reused human) union reaches a perfect 100%; the AI-only and human-only segments are complementary.

2. **Human supremacy in a specific domain:** Uniquely in the Budget Planner, the (reused) human union (95.0%) exceeds the (newly-generated) AI union (92.5%), reflecting the original professional testers' strength in mapping complex budget-calculation state transitions.

New Cohort (Library Fine Calculator, Task Manager)

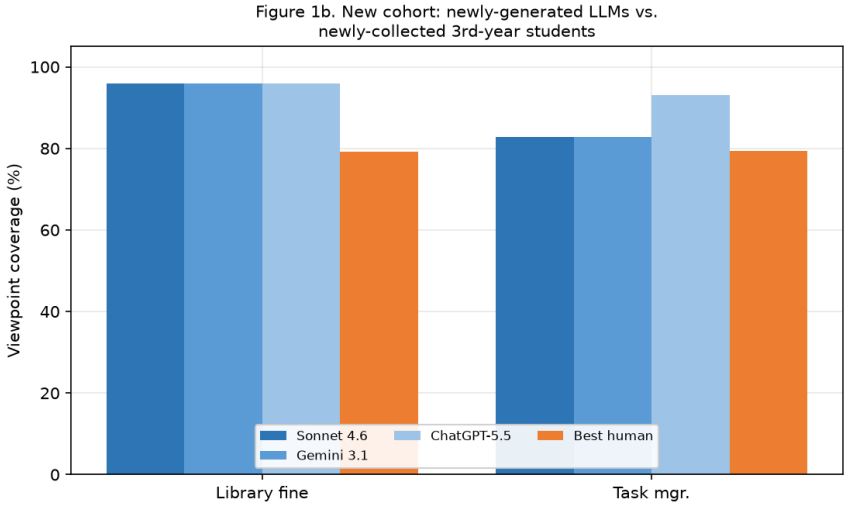

Figure 4: New cohort — newly-generated LLMs (Sonnet 4.6, Gemini 3.1, ChatGPT-5.5) vs. the best of the newly-collected 3rd-year students, per application

Figure 4 shows the same pattern holds in the entirely-original new cohort: newly-generated LLMs substantially outperform the best newly-collected student in both applications, and the gap is markedly larger here than in the reproduced cohort (Task Manager: ChatGPT-5.5 93.1% vs. best student 79.3%) because the student cohort includes one very low-performing outlier (Student C, discussed in Section 6's Expected Contributions).

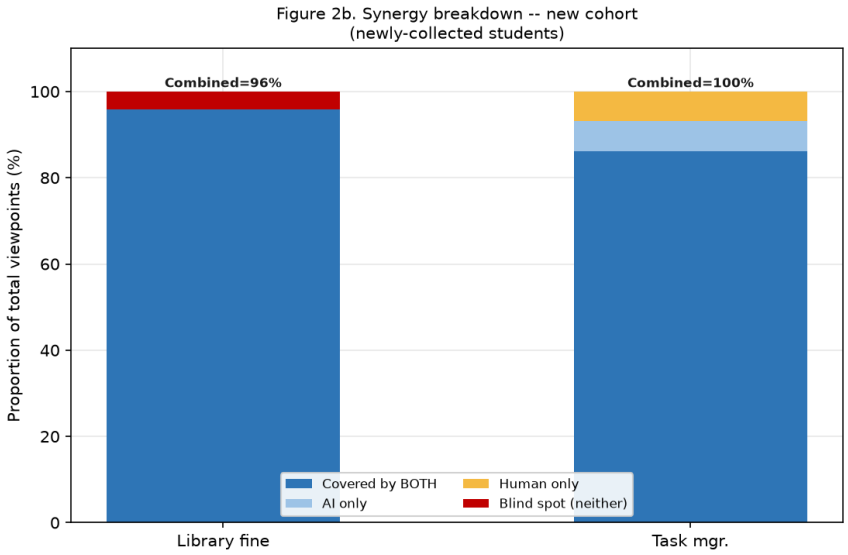

Figure 5: Synergy breakdown — new cohort (newly-collected students)

Figure 5 shows the same four-segment breakdown for the new cohort:

1. **100% combined union (Task Manager):** the newly-generated AI-only and newly-collected human-only segments are complementary, closing the gap to full coverage.

2. **Shared blind spot (Library Fine Calculator):** the combined union is only 96.0%. Exactly **1 viewpoint** was missed by all 3 AI models *and* all 3 student testers — a genuine shared blind spot that Human+AI collaboration cannot recover, because neither source ever covered it.

Pooled Category-Level View (Statistical Use Only)

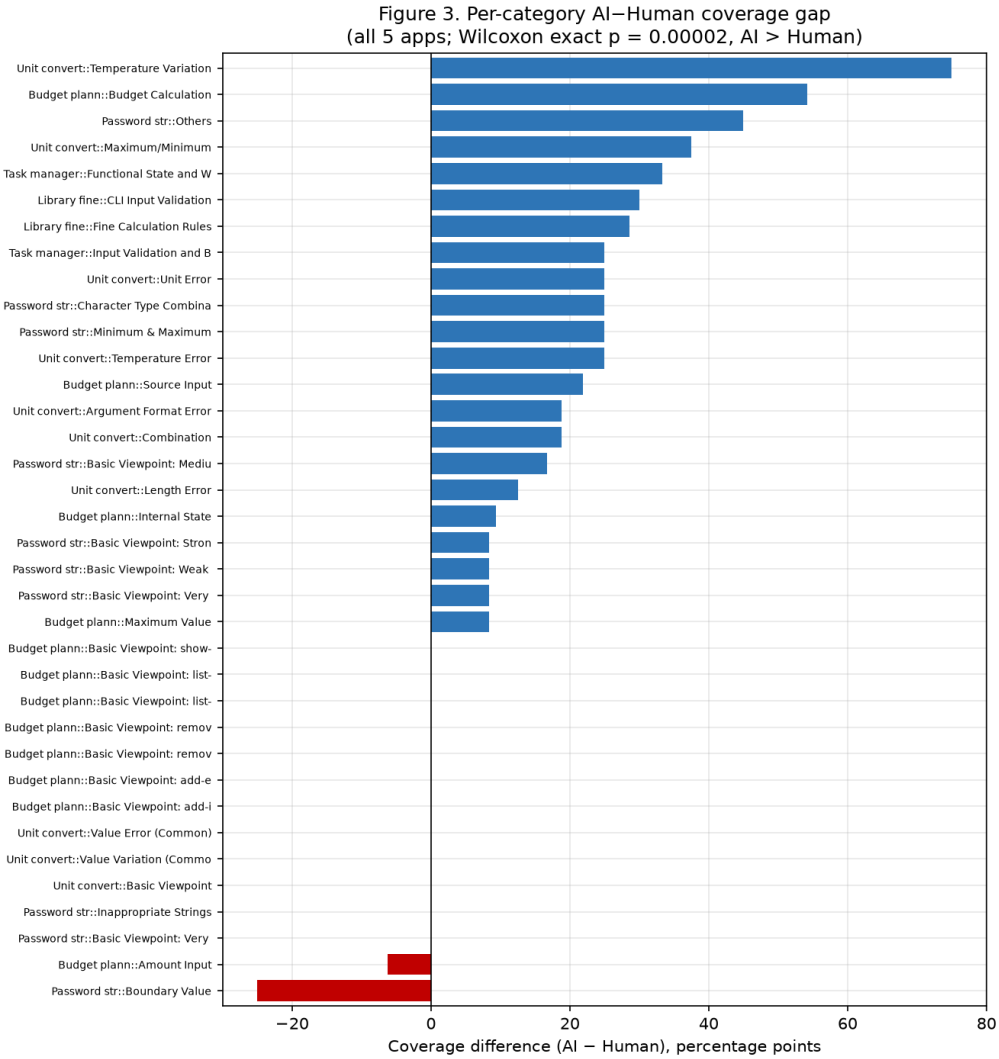

Figure 6: Per-category AI–Human coverage gap, all 24 categories pooled from both cohorts

Unlike Figures 2–5, Figure 6 *intentionally* pools all 24 categories from both cohorts on one axis — but only because this is exactly the pooled sample that feeds the "Category (all 5 apps, N=24)" row of the Wilcoxon test in Table 9 (Section 4.4), where pooling is a deliberate, explicitly-labeled statistical technique to gain test power, not a visual claim that the two cohorts are the same experiment. The two negative bars indicate the only categories (from either cohort) where humans outperformed AI: *Password strength checker :: Boundary Value* (−25 pp, reproduced cohort) and *Budget planner :: Amount Input* (−6.2 pp, reproduced cohort). In contrast, AI substantially outperformed humans in *Unit converter :: Temperature Variation* (+75 pp, reproduced cohort) and *Budget planner :: Budget Calculation* (+54.2 pp, reproduced cohort) — areas of systematic enumeration and multi-condition state-transition reasoning.

Summary Table: Coverage by Cohort

Table 8: Coverage summary by cohort

Cohort	Applications	AI Avg.	Human Avg.	AI Union	Human Union	Combined
Reproduced (professional)	Password, Unit, Budget	81.1%	60.9%	94.3%	90.5%	**100.0%**
New (student)	Library Fine, Task Mgr	89.2%	61.1%	94.3%	94.3%	**98.1%**

Statistical Significance Results

Table 9: Wilcoxon signed-rank test results

Test	N_{usable}	W^+	W^-	Exact p	Conclusion
Application level ($N = 5$)	5	15.0	0.0	0.06250	Not sig. (min. possible for N=5)
Category (reproduced, $N = 20$)	20	194.5	15.5	0.00027	Significant
Category (all 5 apps, $N = 24$)	24	284.0	16.0	0.00002	Highly significant

With only five applications, the minimum achievable exact two-sided p-value is $2^{-4} = 0.0625$; since all five AI means exceeded their human counterparts ($W^- = 0$), we obtained exactly that mathematical floor. This is not a failed test, but a sample-size limitation. Pooling all 24 categories across the five applications yields an exact p-value of 0.00002, with AI $>$ Human in 22 of 24 categories — overwhelming evidence for the primary hypothesis.

Expected Contributions

1. **A validated framework** for comparing LLM and human black-box test suites using reproducible, scripted viewpoint extraction and exact statistical testing.
2. **A characterization of the professional vs. student gap:** student testers showed extreme variance (20.7%–79.3% coverage), suggesting GenAI is especially valuable as a quality floor for teams with inexperienced testers.
3. **A synergistic QA workflow model:** since Human+AI combined coverage reaches $\geq$98.1%, we propose a two-phase process — (1) use LLMs to generate baseline coverage, then (2) use humans for targeted boundary and domain-specific review.

7 Limitations and Risk

7.1 Limitations

Subjectivity in viewpoint extraction. The mapping of raw test-case text to discrete viewpoints in `test perspective analysis 2.xlsx` was done manually. Despite cross-review by two researchers, subjective judgment was involved in deciding whether a test case implicitly "covers" a viewpoint — a construct-validity threat. Future work should explore automated viewpoint extraction using NLP classifiers.

Scale of applications. All five applications are small-scale, terminal-based CLI tools with well-defined specifications. The coverage advantages observed for LLMs may not generalize to large-scale, enterprise systems with implicit requirements or domain-specific jargon.

Viewpoint coverage $\neq$ fault detection. This study measures only whether a tester *addressed* a requirement aspect, not whether the corresponding assertion would actually catch a real bug. As emphasized in the course methodology, coverage answers "did the suite mention this viewpoint?" while a *seeded-fault (mutation) analysis* answers "can the suite catch a wrong implementation?". A natural extension of this study would inject small, single-behavior mutants into the two new CLI specifications — e.g., for the Task Manager, mutants such as *default priority becomes "low" instead of "medium"*, *invalid priority "urgent" is accepted*, or *task IDs start from 0 instead of 1* — and compute a mutation score (killed mutants / total mutants) for each AI and human test suite. This was intentionally left out of the current scope due to the additional implementation and execution effort required, but is identified as the most direct next step to complement the viewpoint-coverage results reported here.

Student cohort size. Only three students participated in the new-cohort experiment. This small sample size means the findings (especially the extreme variance, e.g., Student C at 20.7%) may not generalize to the broader population of third-year software engineering students. A larger cohort ($N \geq 10$) would strengthen external validity.

LLM non-determinism. LLM outputs are non-deterministic. The test suites generated in this study represent a single sampling of each model's output; repeated runs of the same prompt could yield different coverage scores.

Reused-baseline provenance (reproduced cohort only). The Human A–D and ChatGPT (GPT-4) suites for the reproduced cohort are not freshly collected by this team; they are inherited unchanged from a study conducted in a different year, under a protocol this team did not control. Pairing that historical human/GPT-4 data with newly generated GPT-5.5/Sonnet 4.6/Gemini 3.1 suites is methodologically sound for viewpoint coverage (both sides are scored against the same fixed specification and rubric), but it does introduce a mild temporal-consistency threat that does not apply to the new cohort, where every suite — AI and human alike — was generated under identical, contemporaneous conditions. This is precisely why the two cohorts are never pooled into a single statistic in this report.

7.2 Risk and Mitigation Strategies

Table 10: Risk analysis and mitigation strategies

Risk	Analysis	Impact	Mitigation
Publication bias	LLM research tends to over-report positive results; selected papers may inflate expected coverage benchmarks.	Medium	Measurements were cross-checked directly from the raw test-case files against the Excel matrix, independently of published claims.
Column-mapping errors in the Excel parser	Initial scripts mis-mapped *Gemini 3.1* to a blank column instead of its actual column, corrupting statistics for Password and Unit Converter.	High	A full manual audit of all five sheet structures was performed and all column mappings were corrected; results were regenerated from scratch.
Heterogeneous human baseline & data provenance	Mixing professional (reused, secondary) and student (newly collected, primary) data without distinguishing cohorts would produce a misleading average human benchmark and obscure that only 3 of the 5 applications' human data was collected by this team.	High	The two cohorts are strictly separated throughout all analyses, tables, and figures, and are reported independently at every stage; no pooled "AI vs. human" statistic spans both cohorts.
Overfitting the Wilcoxon test to category selection	Hand-picking the granularity of the test (application vs. category level) could introduce data-dredging concerns.	Medium	All three test levels are reported transparently, including the non-significant application-level test; no cherry-picking of results.
Prompt engineering bias	Different prompt templates could systematically advantage certain models.	Low	A single, standardized zero-shot prompt template was used for all four LLMs.

8 Specification Collection from Zenodo

To prepare the input artifacts for the test-case generation phase, the study used the replication materials associated with Kirinuki & Tanno [1]. In their paper, the authors state that the application specifications, ChatGPT prompts, generated test cases, and extracted test viewpoints were shared via Zenodo at https://zenodo.org/records/10476924. This record was used as the primary source for two things, both limited to the reproduced cohort: (i) the software specifications of the three reproduced applications (Password Strength Checker, Unit Converter, Budget Planner), and (ii) the original human (Human A–D) and ChatGPT (GPT-4) test suites for those same three applications, which are reused unchanged as the fixed baseline described in Section 4.2. Only the specifications were needed as prompt input; the original human and ChatGPT-4 test cases were not regenerated or re-run.

The specification-collection process consisted of three steps. First, the team accessed the Zenodo record and identified the available specification files for the subject applications. Second, the collected specifications were reviewed and reformatted into a consistent requirement structure so they could be used as clear prompt inputs for the three *newly added* LLMs (GPT-5.5,

Sonnet 4.6, Gemini 3.1) — the fourth model, ChatGPT (GPT-4), already had its test cases published in the same record and did not need to be re-prompted. Third, all AI-generated test cases (the three new ones, plus the reused GPT-4 set) were mapped back to the original specifications to evaluate requirement coverage, viewpoint coverage, boundary-value scenarios, and possible hallucinated or unsupported expected results.

For the two new CLI applications introduced in this study (Library Fine Calculator, Task Manager), no equivalent Zenodo materials existed; original specifications were authored by the team specifically to extend the replication study beyond the original three applications, following the same requirement structure used for the reproduced set.

This step is important because the quality of AI-generated black-box test cases depends strongly on the clarity and completeness of the input specification. By anchoring the reproduced cohort to the Zenodo materials connected to the reference paper, the study maintains traceability between the original research source, the extracted specifications, the LLM prompts, and the final test cases used for evaluation.

All specifications, prompts, test-case files, and analysis scripts used in this study are available at: https://github.com/dpvinh30092005/research-testing-model-ai.

References

[1] H. Kirinuki and H. Tanno, "ChatGPT and Human Synergy in Black-Box Testing: A Comparative Analysis," *arXiv preprint arXiv:2401.13924*, 2024.

[2] C. Arora, T. Herda, and V. Homm, "Generating Test Scenarios from NL Requirements using Retrieval-Augmented LLMs: An Industrial Study," *arXiv preprint arXiv:2404.12772*, Apr. 2024.

[3] A. M. Sami, Z. Rasheed, M. Waseem, Z. Zhang, H. Tomas, and P. Abrahamsson, "A Tool for Test Case Scenarios Generation Using Large Language Models," *arXiv preprint arXiv:2406.07021*, Jun. 2024.

[4] C. Tufano et al., "TestGenEval: A Real World Unit Test Generation and Test Completion Benchmark," *arXiv preprint arXiv:2410.00752*, Oct. 2024.

[5] M. Ferreira, L. Viegas, and J. P. Faria, "Acceptance Test Generation with Large Language Models: An Industrial Case Study," in *Proc. IEEE/ACM International Conference on Automation of Software Test (AST)*, 2025. arXiv:2504.07244.

[6] Course Lecture Slides, "Collection & Classification in Research," SWT301 — Software Testing, FPT University, 2026.

A Extracted Test Viewpoints in Each Test Suite

The detailed extracted test viewpoints covered by each test suite for all evaluated applications are presented below. All tables have been styled uniformly for optimal readability. Recall from Section 4.2 that Tables 11–13 (reproduced cohort) contain *reused* Human A–D / GPT-4 columns alongside this study's newly-generated GPT-5.5 / Sonnet 4.6 / Gemini 3.1 columns, while Tables 14–15 (new cohort) contain only newly-generated data (three LLMs and three students).

Table 11: Extracted Test Viewpoints from Password Strength Checker

Category	Test Viewpoint	GPT-5.5	Sonnet	Gemini	GPT-4	A	B	C	D	Eff?
Basic: Very Weak	7 characters or fewer & 1 type of character	✓	✓	✓	✓	✓	✓	✓	✓	✓
	7 characters or fewer & 2 or more types of characters	✓	✓	✓		✓	✓	✓	✓	✓
	8 characters or more & 1 type of character	✓	✓	✓	✓	✓	✓	✓	✓	✓
Basic: Weak	8,9 characters & 2 or 3 types of characters (exc. symbols)	✓	✓	✓	✓	✓	✓	✓	✓	✓
	8,9 characters & 2 or more types of chars including symbols	✓	✓	✓	✓		✓	✓	✓	✓
	10 characters or more & 2 or 3 types of chars (exc. symbols)	✓	✓	✓	✓	✓	✓	✓	✓	✓
Basic: Medium	10,11 characters & 2 types of characters including symbols	✓	✓	✓	✓	✓	✓	✓		✓
	10,11 characters & 3 or more types of chars including symbols	✓	✓	✓	✓		✓	✓	✓	✓
	12 characters or more & 2 types of characters including symbols		✓	✓	✓	✓		✓	✓	✓
Basic: Strong	12∼15 characters & 3 types of characters including symbols	✓	✓	✓	✓	✓	✓	✓		✓
	12∼15 characters & 4 types of characters	✓	✓	✓			✓	✓	✓	✓
	16 characters or more & 3 types of characters including symbols	✓		✓	✓	✓	✓	✓	✓	✓
Basic: Very Strong	16 characters or more & 4 types of characters	✓	✓	✓	✓	✓	✓	✓	✓	✓
Boundary Value	String length boundary values (exc. min and max lengths)	✓		✓		✓	✓	✓	✓	✓
Minimum & Maximum	Minimum length (1)	✓	✓	✓				✓		
	Maximum length (100)	✓	✓	✓	✓	✓	✓	✓	✓	✓
Character Type Comb.	4 ways to choose 1 type of character	✓	✓	✓	✓		✓	✓	✓	✓
	3 combinations of 2 types of characters excluding symbols	✓	✓	✓	✓		✓	✓	✓	✓
	3 combinations of 2 types of characters including symbols	✓	✓	✓	✓		✓	✓	✓	✓
	3 combinations of 3 types of characters including symbols	✓	✓	✓	✓		✓	✓	✓	✓
Inappropriate Strings	Failing by not providing a string	✓	✓	✓	✓	✓	✓		✓	✓

Table 12: Extracted Test Viewpoints from Unit Converter

Category	Test Viewpoint	GPT-5.5	Sonnet	Gemini	GPT-4	A	B	C	D	Eff?
Basic Viewpoint	For all units of length, appearing in either source or target...	✓	✓	✓	✓	✓	✓	✓	✓	✓
	For all units of temperature, appearing in either source or target...	✓	✓	✓	✓	✓	✓	✓	✓	✓
Combination	Exhaustive coverage of two-unit combinations for length	✓	✓	✓	✓	✓	✓	✓	✓	✓
	All units of length appear in both source and target	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Exhaustive coverage of two-unit combinations for temperature	✓	✓	✓	✓	✓	✓		✓	✓
	All units of temperature appear in both source and target	✓	✓	✓	✓	✓	✓	✓	✓	✓
Unit Error	Conversion to a unit from a different category	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Conversion between the same units	✓	✓	✓	✓		✓	✓		✓
	Providing a length unit that is not supported	✓	✓	✓	✓		✓	✓		✓
Value Variation	When the pre-conversion value is an integer	✓	✓	✓	✓	✓	✓	✓	✓	✓
	When the pre-conversion value is a decimal	✓	✓	✓	✓	✓	✓	✓	✓	✓
Value Error	Providing a value exceeding the maximum value	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Providing a value up to the third decimal place	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Providing an invalid character in the pre-conversion value	✓	✓	✓	✓	✓	✓	✓	✓	✓
Length Error	When the pre-conversion value for length is negative	✓	✓	✓	✓	✓	✓		✓	✓
	When the pre-conversion value for length is zero	✓	✓	✓		✓	✓	✓	✓	✓
Temperature Variation	Successful conversion when pre-conversion value is zero	✓	✓	✓				✓		
	Successful conversion when pre-conversion value is negative	✓	✓	✓	✓					
Temperature Error	When the pre-conversion value in Kelvin is zero	✓	✓	✓	✓				✓	✓
	When the pre-conversion value in Kelvin is negative	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Giving a value smaller than the minimum for Celsius/Fahrenheit	✓			✓			✓		✓
Argument Format Error	When no value is provided	✓	✓	✓	✓		✓	✓	✓	✓
	When no 'from' is provided	✓	✓		✓			✓	✓	✓
	When no 'to' is provided	✓	✓		✓			✓	✓	✓
	When multiple inputs are made for from, value, and to								✓	

Table 13: Extracted Test Viewpoints from Budget Planner

Category	Test Viewpoint	GPT-5.5	Sonnet	Gemini	GPT-4	A	B	C	D	Eff?
Basic Viewpoint	Successfully add an income	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully add an expense	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully remove an income	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully remove an expense	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully display incomes	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully display expenses	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Successfully display the budget	✓	✓	✓			✓	✓	✓	✓
Amount Input	Giving 0	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Giving a negative number	✓	✓	✓	✓		✓	✓	✓	✓
	Not providing an amount	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Giving a decimal number	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Giving a value exceeding the maximum value	✓	✓	✓		✓	✓	✓	✓	✓
	Giving a non-numeric value	✓	✓	✓		✓				
	Giving a multibyte/Unicode character								✓	
	Giving multiple amounts		✓		✓	✓	✓	✓	✓	✓
Source Input	Not providing a source	✓	✓	✓	✓	✓		✓		✓
	Giving an empty string	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Exceeding the maximum string length	✓	✓	✓					✓	
	Giving a non-string value				✓					
	Giving a multibyte/Unicode character	✓			✓					
	Including a space	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Not enclosing in double quotations					✓				
	Giving multiple sources	✓	✓	✓	✓	✓	✓		✓	✓
Internal State	Exceeding the maximum number of registrations		✓		✓	✓	✓	✓	✓	✓
	Deleting a non-existent source	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Displaying when there is no source	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Registering multiple sources	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Verifying that deletion is reflected in the state	✓	✓	✓	✓	✓	✓	✓	✓	✓
	Registering with the same name for income or expense	✓	✓	✓				✓		
	Registering an expense source with same name as an income	✓	✓	✓	✓	✓		✓		✓
	Deleting an income source as an expense (and vice versa)	✓				✓		✓		✓
Maximum Value	Maximum value for amount	✓	✓	✓	✓	✓		✓		✓
	Maximum string length	✓	✓	✓	✓	✓	✓	✓		✓
	Maximum number of registrations		✓		✓	✓	✓	✓	✓	✓

Table 14: Extracted Test Viewpoints from Library Fine Calculator

Category	Test Viewpoint	Sonnet	Gemini	GPT-5.5	A	B	C	D	Eff?
Fine Calculation Rules	Calculate book overdue fine below the cap for a non-member	✓	✓	✓	✓	✓	✓		✓
	Calculate dvd overdue fine below the cap for a non-member	✓	✓	✓	✓	✓	✓		✓
	Apply book overdue fine cap at $20.00	✓	✓	✓	✓	✓	✓		✓
	Apply dvd overdue fine cap at $30.00	✓	✓	✓	✓	✓	✓		✓
	Apply 50% member discount to uncapped overdue fine	✓	✓	✓	✓				✓
	Apply 50% member discount to capped overdue fine	✓	✓	✓		✓	✓		✓
	Verify member discount does not apply to lost-item fee	✓	✓	✓	✓	✓	✓		✓
	Add book replacement fee when –lost yes	✓	✓	✓	✓		✓		✓
	Add dvd replacement fee when –lost yes	✓	✓	✓					✓
	Calculate combined cap, member discount, and lost fee for book	✓	✓	✓		✓			✓
	Calculate combined cap, member discount, and lost fee for dvd	✓	✓	✓		✓	✓		✓
	Return $0.00 when days-late is 0 and item is not lost	✓	✓	✓	✓	✓	✓		✓
	Return only the replacement fee when days-late is 0 and lost	✓	✓	✓	✓	✓			✓
	Output all successful totals with exactly two decimal places	✓	✓	✓	✓	✓	✓		✓
CLI Input Validation	Reject missing required option	✓	✓	✓	✓	✓	✓		✓
	Reject unknown options and duplicate options	✓	✓	✓					
	Reject invalid item type and case-mismatched type values	✓	✓	✓	✓	✓	✓		✓
	Accept days-late boundary values 0 and 365	✓	✓	✓	✓	✓	✓		✓
	Reject days-late below 0 and above 365	✓	✓	✓	✓	✓	✓		✓
	Reject decimal days-late values	✓	✓	✓	✓	✓			✓
	Reject non-numeric days-late values	✓	✓	✓		✓			✓
	Reject invalid member status and case-mismatched values	✓	✓	✓		✓			✓
	Reject invalid lost status and case-mismatched values	✓	✓	✓		✓			✓
	Verify each invocation is stateless								

Table 15: Extracted Test Viewpoints from Task Manager

Category	Test Viewpoint	Sonnet	Gemini	GPT-5.5	A	B	C	D	Eff?
Functional State & Workflow Behavior	Successfully add a task with required fields only	✓	✓	✓	✓	✓	✓		✓
	Successfully add a task with explicit low, medium, high priority	✓	✓	✓	✓	✓			✓
	Verify default priority is medium when –priority is omitted	✓	✓	✓	✓	✓	✓		
	Automatically assign task IDs starting from 1 in creation order	✓	✓	✓		✓	✓		✓
	Update only the title of an existing task	✓	✓	✓		✓			✓
	Update only the due date of an existing task	✓		✓		✓			✓
	Update only the priority of an existing task	✓	✓	✓		✓			✓
	Update multiple editable fields in one command	✓	✓	✓	✓	✓	✓		✓
	Update a completed task without changing its done status	✓	✓	✓		✓			
	Complete a todo task and reflect the done status in list output	✓	✓	✓	✓	✓			✓
	Reopen a done task and reflect the todo status in list output	✓	✓	✓	✓				✓
	Delete todo and done tasks and verify they disappear	✓	✓	✓					✓
	Verify deleted task IDs are not reused after adding a new task	✓	✓	✓	✓	✓	✓		✓
	List all tasks, only todo tasks, and only done tasks	✓	✓	✓	✓	✓			✓
	Sort tasks by created order, due date, and priority severity		✓	✓	✓	✓			✓
	Combine status filtering with sorting	✓		✓		✓			
	Show summary counts after operations		✓	✓	✓	✓			
Input Validation & Boundary Behavior	Reject missing command, unknown command, and option	✓	✓	✓					
	Reject blank title, 0-character title, and title > 80 characters			✓	✓				✓
	Accept minimum and maximum valid title length	✓	✓	✓	✓	✓			✓
	Reject title containing characters outside the allowed set	✓	✓	✓		✓			✓
	Reject invalid due date format and invalid real calendar dates	✓	✓	✓	✓	✓	✓		✓
	Verify leap-year boundary dates, including valid 2000-02-29	✓	✓	✓	✓	✓			✓
	Reject due date year below 2000 and above 2099	✓	✓	✓	✓	✓			✓
	Reject invalid or case-mismatched priority, status, sort values	✓	✓	✓	✓				✓